

Событийно-ориентированная архитектура (EDA) для AI

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Фомин Дмитрий

Руководитель отдела архитектуры

- Больше 15 лет опыта работы архитектором
- Участие в разработке архитектуры в очень больших проектах (BSS)

 @advoc_diaboly

 fomin.dmitry@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#OTUS**
AI-Architect-2026-04



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Смена парадигмы

Архитектурные паттерны EDA

Event Sourcing и CQRS

Пример

Основные тезисы

Цели вебинара

К концу занятия вы сможете

проектировать асинхронные,
слабосвязанные системы на основе
событий

применять EDA для построения
масштабируемых AI-пайплайнов и
систем принятия решений

Смысл

Для чего вам это уметь

для того, чтобы ваши системы были
устойчивы и производительны



**Что общего у синхронных и
асинхронных интеграций?**



Смена парадигмы: От запросов к потокам событий

Почему синхронность убивает масштабируемость

- Связность: В синхронной системе Producer обязан знать адрес и API Consumer'a.
- Временная связность: Оба сервиса должны быть доступны одновременно.
- Проблема для AI: Инференс тяжелых моделей (BERT, LLM) занимает время. Блокировать клиентский запрос на 500мс+ — дорого и рискованно.
- Решение: Инверсия контроля через события.



Архитектура, управляемая событием (EDA)– это

это современный архитектурный шаблон, созданный из несвязанных сервисов, которые публикуют, потребляют или маршрутизируют события.

Event Driven Architecture (EDA)

- В управляемой событиями архитектуре, когда микросервис выполняет действие, в котором потенциально заинтересованы другие микросервисы, он публикует событие в брокере событий. Этот микросервис называется publisher (продюсеры).
- Другие микросервисы в системе, называемые consumer (потребители), могут обрабатывать событие.
- EDA представляет собой распределенную систему. Это означает, что компоненты системы расположены на разных серверах и взаимодействие происходит по сети.



Event Driven Architecture - Преимущества

- Слабая связанность
 - Производители и потребители не знают друг о друге, поскольку брокер событий выступает в качестве посредника
- Масштабирование
 - Горизонтальное масштабирование продюсеров и/или потребителей из-за слабой связанность и асинхронного характера событий
- Отказоустойчивость
 - Если при обработке события сервис неожиданно прекращает работу, событие не подтверждается брокером событий как успешно обработанное. Событие не теряется, а вместо этого повторно используется другим экземпляром потребителя или обработается снова, когда сервис снова станет доступным



Event Driven Architecture - Проблемы

- Понимание всего бизнес-процесса
 - Слабая связанность продюсеров и потребителей затрудняет быстрое понимание того, как реализован процесс. Просмотр исходного кода сервиса не дает ответа на этот вопрос.
- Проектирование под отказ
 - В распределенных системах из-за ненадежной сети, нужно корректно обрабатывать повторяющиеся события (идемпотентность), а также обрабатывать сообщения не по порядку (коммутативность)



Вопросы



если есть вопросы



если вопросов нет

Архитектурные паттерны EDA

Event vs Command vs Query

- Событие (Event) – это «факт» – сообщение о том, что произошло изменение состояния объекта или его обновление. Требование от другого сервиса отсутствует.
 - ProductAdded, OrderCreated
- Команда (Command) - это «глагол» – требование что-то сделать от другого сервиса, и ожидается, что конкретный получатель отреагирует
 - AddProduct, CreateOrder
- Запрос – получение внутреннего состояния объекта и возврат его вызывающей стороне. Запрос не вызывает никаких изменений состояния в системе. Наиболее распространенной реализацией запроса является GET-запрос или запрос к базе данных для получения данных.
 - getProduct, getOrder



Что внутри?

- JSON
- XML
- Protobuf
- Thrift
- Avro

```
message Person {  
  required string name = 1;  
  required int32 id = 2;  
  optional string email = 3;  
}
```

```
struct ListBonks {  
  1: list<Bonk> bonk  
}  
struct NestedListsBonk {  
  1: list<list<list<Bonk>>> bonk  
}
```

```
struct BoolTest {  
  1: optional bool b = true;  
  2: optional string s = "true";  
}
```

```
{  
  "namespace": "example.avro",  
  "type": "record",  
  "name": "User",  
  "fields": [  
    {  
      "name": "name", "type": "string",  
    },  
    {  
      "name": "favorite_number", "type": ["int", "null"],  
    },  
    {  
      "name": "favorite_color", "type": ["string", "null"],  
    },  
  ]  
}
```

- Thin Event
 - Передаем только ID
Получатель сам идет в БД за данными.
 - Минус: нагрузка на БД
- Fat Event:
 - событие об изменении конкретных данных (specific changes)
 - снимок конкретной всей записи (snapshot record)
 - Плюс: автономность
 - Минус: размер сообщения

Паттерн "Издатель-Подписчик"

- Принцип: Один издатель — много подписчиков. Издатель не знает, кто его слушает.
- Слабая связанность: Мы можем добавить новый AI-сервис (например, "Анализ тональности"), не меняя код загрузчика файлов. Просто подписываемся на тот же топик.
- Отличие от Queue: В очереди сообщение читает один консьюмер. В Pub/Sub — все активные подписчики получают копию.



Паттерн "Dead Letter Queue"

- Проблема: Сообщение, которое крашит парсер или модель (например, битый PDF или атака на модель).
- Риск: Без DLQ такое сообщение будет вечно крутиться в цикле (Infinite Loop), забивая ресурсы.
- Решение: После N попыток изолируем сообщение в DLQ для ручного разбора.



Выбор брокера (RabbitMQ vs Kafka)

Характеристика	RabbitMQ	Kafka
Модель	Очередь задач (Smart Broker)	Лог событий (Dumb Broker)
Доставка	Удаляется после прочтения (Ack)	Хранится N дней (Replay)
Сложность	Средняя	Высокая
Производительность	Средняя	Очень высокая
AI Use Case	Inference Queue (распределение задач на воркеры)	Training Data / Logs (сбор данных для обучения)



Главные проблемы распределённых систем



Mathias Verraes

@mathiasverraes

...

There are only two hard problems in distributed systems: 2. Exactly-once delivery 1. Guaranteed order of messages 2. Exactly-once delivery

[Перевести пост](#)

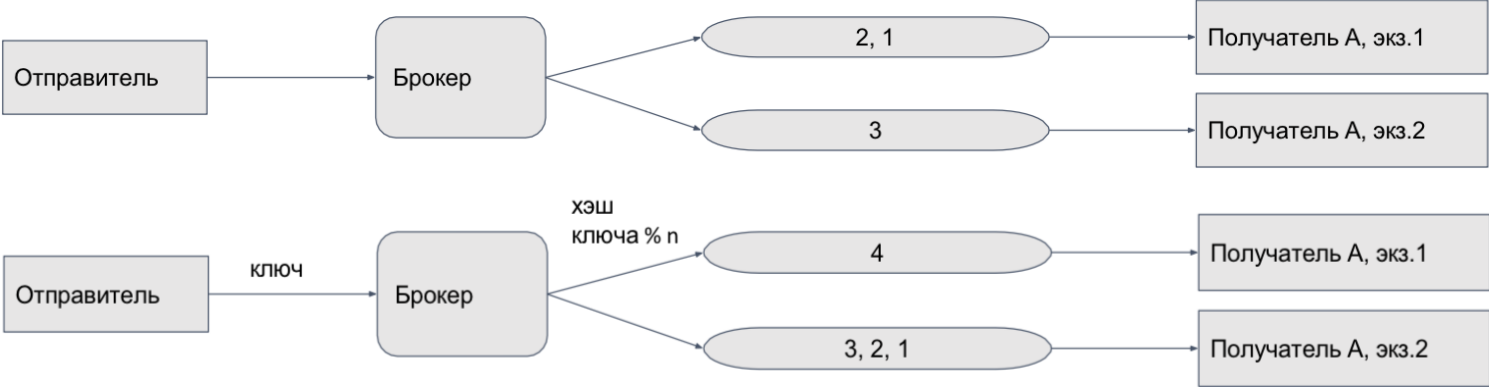
9:40 PM · 14 авг. 2015 г.

- Многие брокеры событий по умолчанию предлагают обработку по крайней мере один раз, что означает, что если брокер не получит подтверждения об успешном использовании события, он повторно отправит событие.
 - Обработка должна быть идемпотентной
- А еще, часто Вам придётся постараться для того, чтобы получать события в правильном порядке (а еще определить - что такое **правильный** порядок)
 - Брокер должен знать ключ, по которому будет определена секция с поддержкой FIFO

Дублирование сообщений – дедупликация



Порядок следования событий

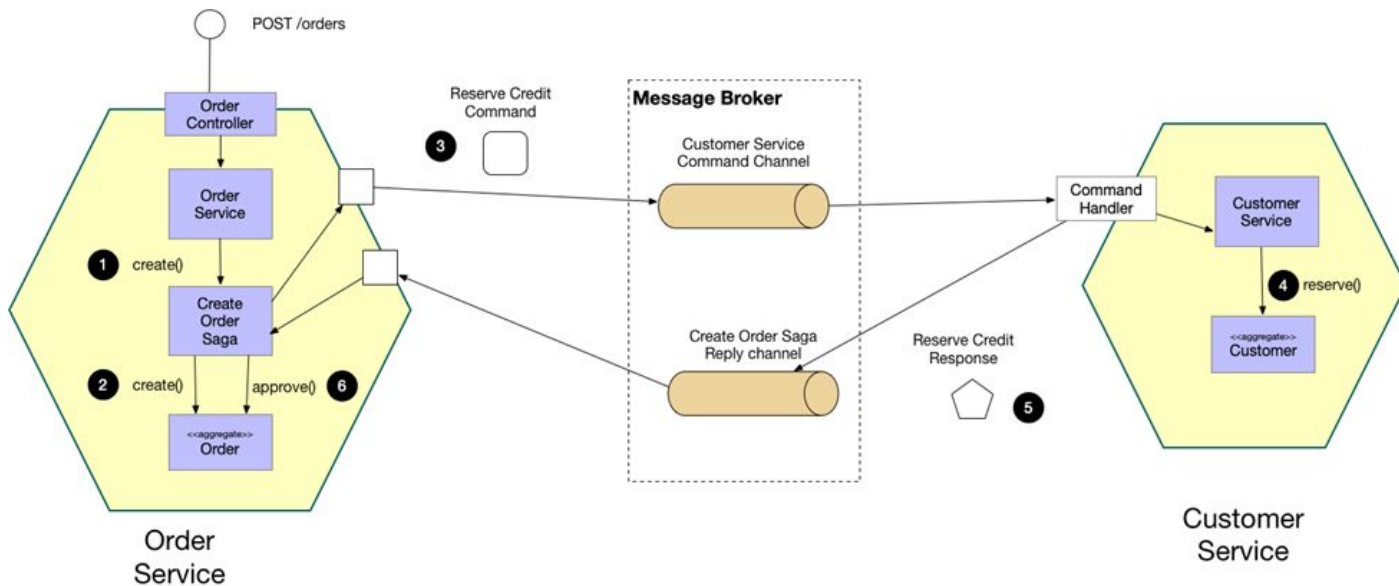


Идентификатор события	Событие	Ключ агрегата
1	OrderCreated	101
2	ProductAdded	101
3	OrderCancelled	101
4	OrderCreated	102



Оркестрация (DAG)–

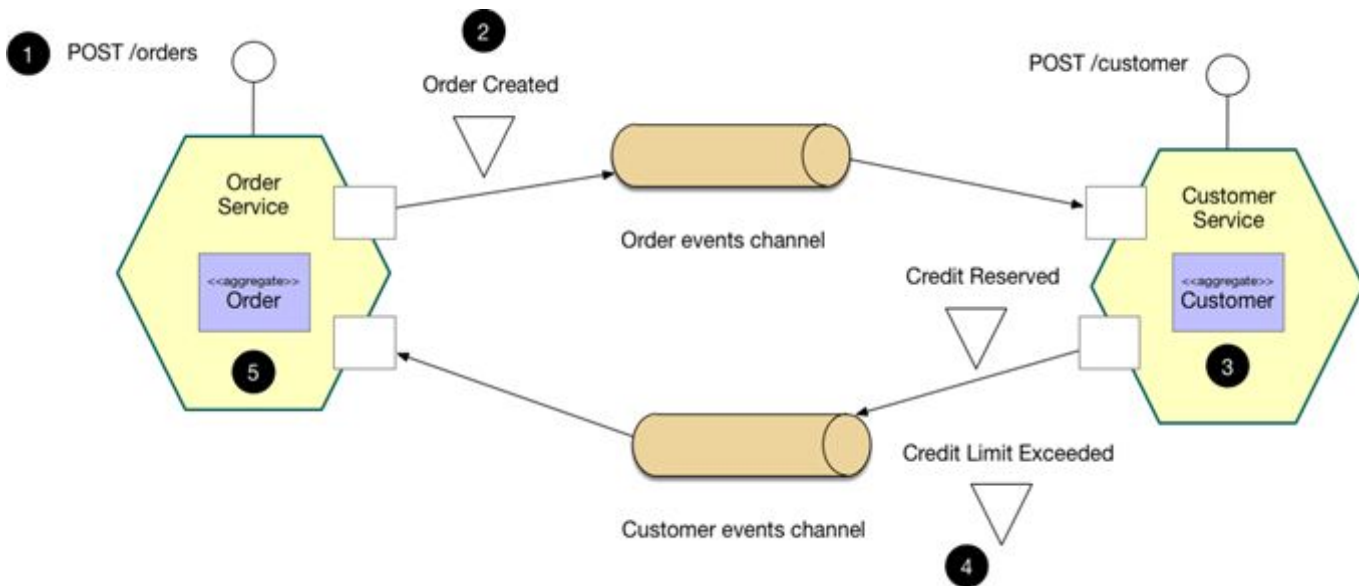
оркестратор (объект) сообщает участникам, какие локальные транзакции выполнять.



<https://microservices.io/patterns/data/saga.html>

Хореография (EDA)–

каждая локальная транзакция публикует доменные события, которые запускают локальные транзакции в других сервисах.



<https://microservices.io/patterns/data/saga.html>

Вопросы



если есть вопросы



если вопросов нет

Паттерны проектирования событий

DataCentric Events– это

это паттерн проектирования при котором под событиями обычно подразумеваются события изменения сущностей в БД (OrderCreated, OrderUpdated).



Проектирование событий

- У пользователя поменялся статус active на blocked у пользователя.
 - Какое событие отправлять?
 - UserUpdated? UserBlocked? UserStatusChanged?
 - Или может быть все вместе?
- Что отправлять в payload такого события?
 - Только изменение? Полностью новое состояние?
 - Только id?
- Блокирование пользователя приводит к блокированию всех его кампаний и баннеров. Сервис, который блокирует кампании и баннеры пользователя должен отослать одно событие? Или несколько по количеству объектов?

Проектирование событий

- На все эти вопросы нет однозначного ответа. Также как нет однозначного ответа на то, каким должен быть REST интерфейс.
 - Должен быть метод PATCH /api/v1/users/{userid}
 - Или POST /api/v1/users/{userId}/block?
 - Или POST /api/v1/users/update?

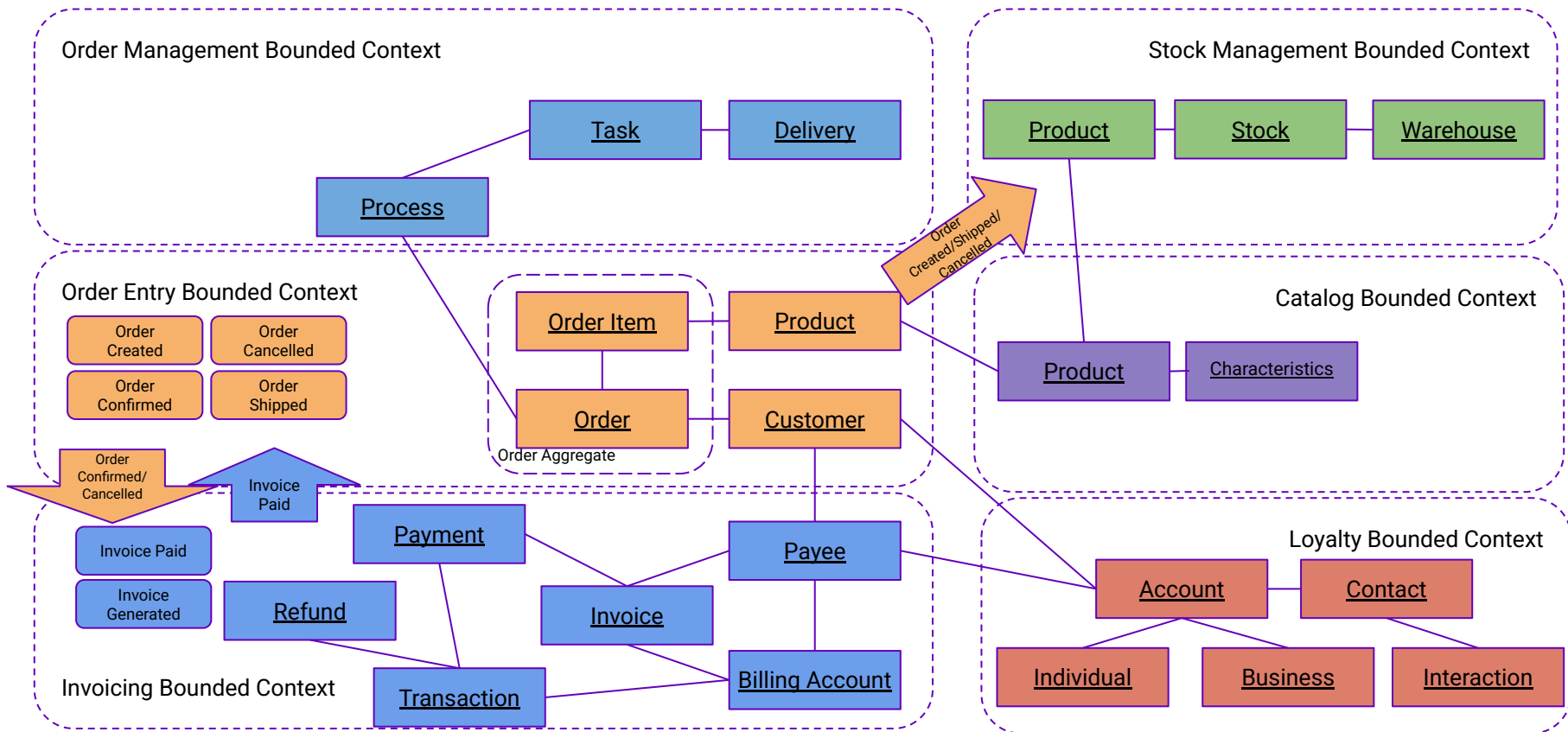
Domain Events– это

это паттерн проектирования при котором под событиями обычно подразумеваются события предметной области – т.е. факты о том, что случилось с сущностями предметной области (OrderSubmitted, OrderProcessed).

С чего начинать проектирование событий?

- Стоит отталкиваться от предметной области.
- Начинать с основного пользовательского сценария.
- Каждое событие стоит связывать с предметной областью.
- На первых итерациях лучше не думать про реализацию на бекенде.

Domain Events как часть DDD



Domain vs Data-Centric

- Data-Centric (CRUD-события)
 - Отражают физическое изменение данных в базе (UserUpdated, RowInserted).
 - Преимущество: Просты в генерации (реализуются через Change Data Capture).
 - Недостаток: Полная утрата бизнес-контекста. Потребитель вынужден вычислять разницу состояний (diff).
- Domain Events (Бизнес-события)
 - Отражают свершившийся факт в терминах предметной области (FraudScoreExceeded, AccountBlocked).
 - Преимущество: Несут явную семантику и намерение.
 - Недостаток: Требуют аналитики и осознанного проектирования на стороне источника.



Вопросы



если есть вопросы

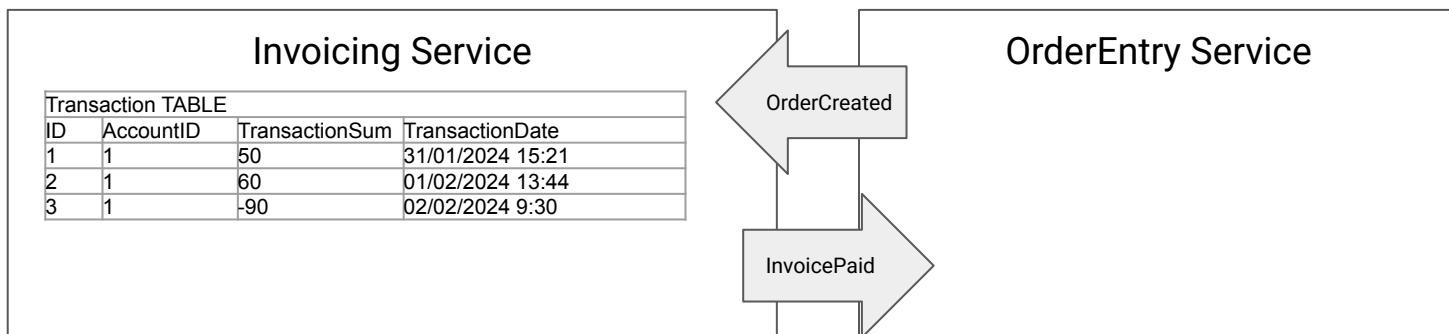


если вопросов нет

Event Sourcing и CQRS

Event Sourcing– это

это паттерн проектирования, при котором в БД хранятся только события, а не состояния объекта



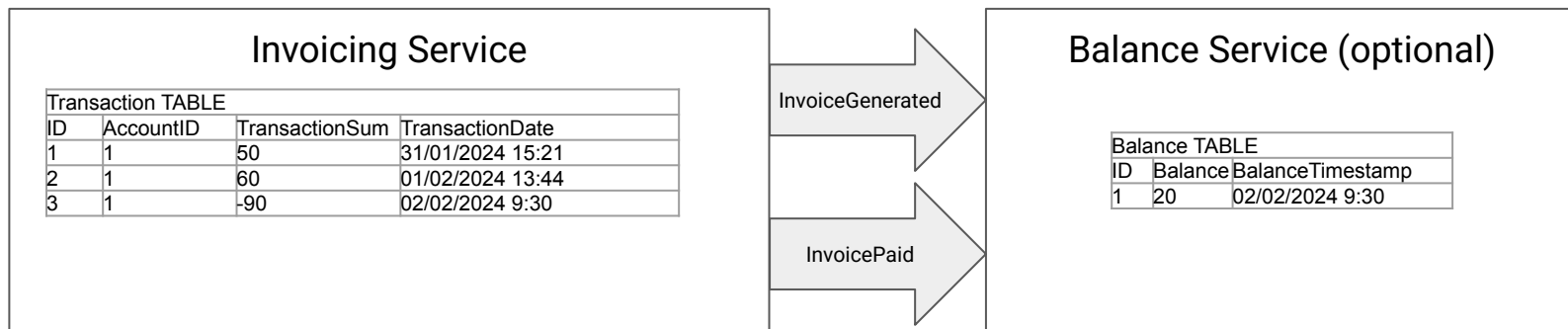
Плюсы и минусы Event Sourcing

- Плюсы
 - Бесплатный аудит-лог
 - Производительность event-store-ов обычно хорошая
 - Можно делать темпоральные запросы (на какой-то момент времени)
 - Можно строить какую угодно аналитику, т.к. храним все данные
- Минусы
 - Миграции данных делать тяжело
 - Возможно резкий рост объемов и как следствие падение производительности
 - Можно решить снапшотами данных
 - Делать выборки (quering) сложно
 - Можно решить при помощи CQRS



CQRS – это

это паттерн проектирования, при котором
разделяются модели данных для модификации и
чтения



Хранение событий: Log Broker vs Event Store

- Log-based брокеры (Apache Kafka)
 - Назначение: Транспорт, буферизация, последовательное массовое чтение.
 - Механика: Ограниченное время хранения (retention), чтение по смещениям (offsets).
 - Ограничение: Отсутствие индексов. Невозможность эффективного поиска истории по одному агрегату.
- Event Store
 - Назначение: База данных для реализации паттерна Event Sourcing.
 - Механика: Хранение независимых потоков (streams) по сущностям, защита от конфликтов через оптимистичную блокировку (Optimistic Concurrency).
 - Преимущество: Нативная поддержка проекций и чтение истории конкретного объекта.
- Стратегия интеграции:
 - Kafka применяется для высоконагруженной маршрутизации. Event Store — как «золотой источник» (Source of Truth) для задач аудита и Explainable AI (XAI).



Вопросы



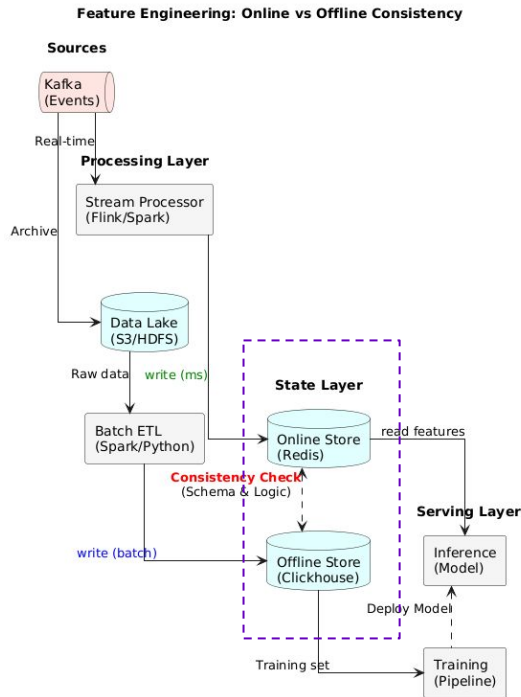
если есть вопросы



если вопросов нет

EDA и AI

Real-time Feature Engineering



- Проблема Batch-подхода: Если считать агрегаты (например, «число покупок за сутки») раз в ночь, то модель не увидит мошенническую атаку, начавшуюся 5 минут назад.
- Решение — Streaming Aggregation:
 - Вычисление фичей «на лету» (On-the-fly) с использованием оконных функций Пример: `count(transactions) over (partition by user_id range interval '1 hour')`.
- State Layer выступает как feature store:
 - EDA позволяет наполнять Feature Store в реальном времени.
- Training-Serving Skew: Главный риск — когда логика расчета фичей в стриминге (Java/Scala) отличается от аналитики (Python/Pandas).

Реактивные паттерны обучения

- Static vs Dynamic:
 - Static: Cron-job «Переобучать каждое воскресенье». Неэффективно, если данные не менялись, и опасно, если появился новый тип мошенничества.
 - Dynamic: Запуск пайплайна по событию-триггеру.
- Типы триггеров (Events):
 - Data Drift Detected: Статистическое распределение входных данных изменилось (например, средняя сумма транзакции выросла в 2 раза).
 - Concept Drift Detected: Точность модели (Accuracy/F1) на свежих данных упала ниже порога.
 - Labels Arrived: Накоплено N новых размеченных примеров (например, 10 000 подтвержденных фрод-кейсов).
- Асинхронная оценка: Получение «истины» (Ground Truth) часто отложено во времени (клиент оспорил транзакцию через 2 дня). Это тоже событие (LabelDelayed), которое должно попасть в Store.

Топология Inference Pipeline

- Вариант 1: Embedded Model (Модель-библиотека)
 - Как работает: Модель (ONNX, POJO) загружается прямо в память Consumer-сервиса (Kafka Consumer).
 - Плюсы: Нулевой сетевой оверхед, максимальная пропускная способность.
 - Минусы: Тяжело масштабировать отдельно от консьюмера, сложно обновлять (нужен редеплой сервиса), конфликт ресурсов (CPU/RAM).
- Вариант 2: External Model Service (RPC / Sidecar)
 - Как работает: Consumer получает событие, собирает фичи и делает gRPC/REST запрос
 - Плюсы: Четкое разделение ответственности, независимый скейлинг GPU-подов.
 - Минусы: Сетевые задержки (Network Latency), риск синхронной блокировки обработки очереди.
- Вариант 3: Asynchronous Inference (Fully EDA)

Вопросы



если есть вопросы



если вопросов нет

Пример

Fraud Monitoring. Бизнес-требования

- Продукт: Система real-time fraud monitoring
- Бизнес-цель: Блокировать подозрительные операции до списания средств.
- NFR:
 - Latency: Строго < 200 мс на весь цикл (от получения запроса до ответа терминалу), в случае задержки валидации транзакция отправляется на постобработку
 - 10,000 TPS (транзакций в сек) в пике.
- Жесткие ограничения (Constraints):
 - Модель требует контекст - набор предрасчитанных агрегатов за последние 30 дней (средний чек, география покупок).
 - «Сырая» транзакция бесполезна без предрасчитанных агрегатов по истории.

Анализ Альтернатив

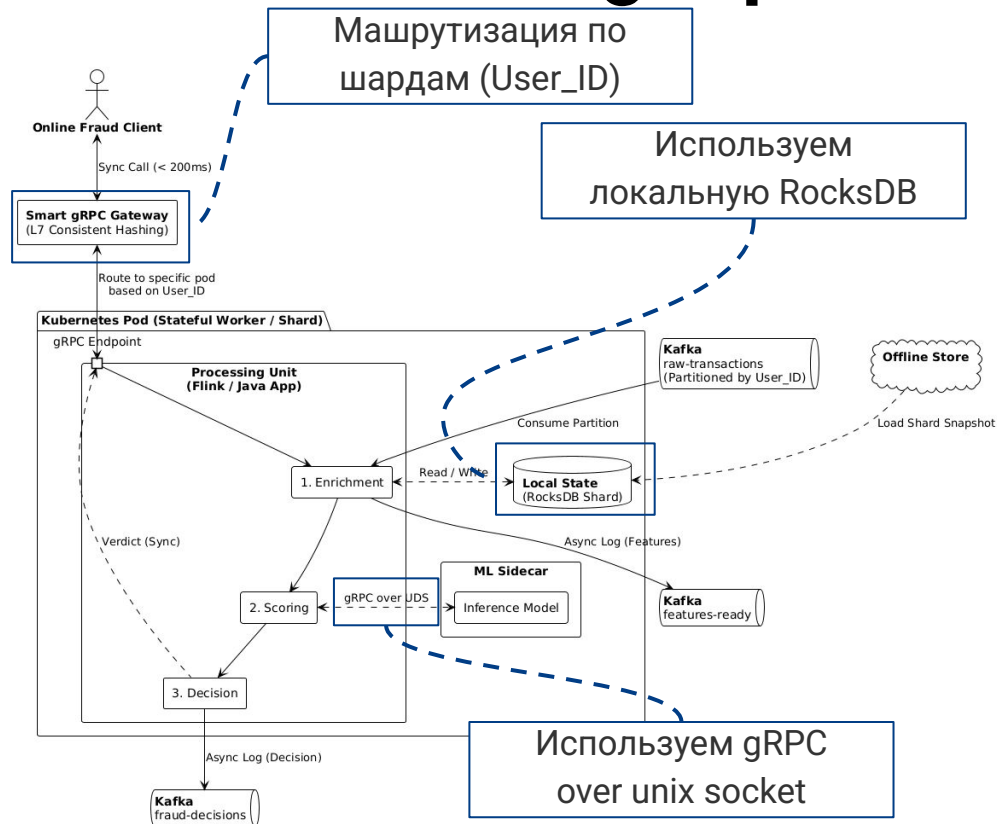
Критерий	Вариант А: Pure EDA (Async)	Вариант Б: Sidecar / Direct (Sync)
Задержка	Требуется две записи в Kafka и два чтения. Плюс время на репликацию.	Данные передаются через RAM (Unix Socket). Колебания задержки минимальны. Доступна интеграция через kafka (near-online) и gRPC (online)
Обработка ошибок	Если модель упала, сообщение лежит в Kafka. Нужен Dead Letter Queue и отдельный процесс разбора.	Если вызов модели вернул ошибку, Flink сразу применяет Fallback-стратегию (например, "Пропустить по лимитам").
Сложность разворачивания	Независимые сервисы. Обновляем модель — Flink продолжает работать.	Единый Pod. Обновление модели требует рестарта Flink. Сложный Resource Management.



Fraud Monitoring. Архитектурные решения

Тип	Требование / Ограничение	Проблема	Архитектурное решение
Consistency	Строгий порядок событий	В распределенной системе событие может быть нарушен порядок событий	Используем User_ID как ключ партиционирования (Partition Key)
NFR	Latency: Строго < 200 мс	Ходить в pgSQL за историей клиента на каждой транзакции — это DDOS собственной базы и задержка	Шардированные агрегаты клиента хранятся прямо на диске/в памяти воркера Stream Processor'a. Используем маршрутизацию gRPC трафика перед подами
NFR	Latency: Строго < 200 мс	Инфраструктура (Kafka/Flink) обычно на JVM (Java/Scala). Data Scientists пишут модели на Python.	Используем паттерн sidecar. Java и Python живут в одном Pod'e и общаются через gRPC over Unix Domain Socket

Fraud Monitoring. Архитектура



- Используем гибридный рантайм чтобы облегчить работу data science команде
- Используем маршрутизацию и шардирование по User_ID
- Агрегаты клиентов шарда хранится локально на SSD воркера в RocksDB, заполняются при потере данных из offline store
- Данные не покидают пределы оперативной памяти сервера (loopback). Мы получаем скорость почти как у прямого вызова функции, но сохраняем полную изоляцию процессов

Вопросы



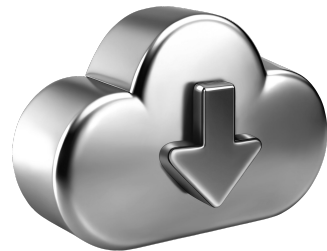
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Laszewski, T. von. 2022. Event-Driven Architecture: A Practical Guide. Apress.
2. Redkar, S. 2021. Event-Driven APIs: A Guide to Building Real-Time, Asynchronous Integrations. Apress.
3. Rogers, A. 2022. Event-Driven Systems: A Practitioner's Guide to Building Real-Time Applications. O'Reilly Media.
4. Фергюсон, Д. 2022. Проектирование событийно-ориентированных систем. Концепции и паттерны. СПб.: Питер.
5. Эспозито, Д. 2023. Современная архитектура программного обеспечения. Паттерны и практики. СПб.: Питер.
6. Янг, Г. 2022. Событийная модель. Проектирование и реализация. М.: ДМК Пресс.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

проектировать асинхронные,
слабосвязанные системы на основе
событий

применять EDA для построения
масштабируемых AI-пайплайнов и
систем принятия решений

для того, чтобы ваши системы были
устойчивы и производительны



Вопросы для проверки

1. Как паттерн Event Sourcing помогает в задачах Explainable AI (объяснимость решений)?
2. В чем риск использования «Тонких событий» (Thin Events, передача только ID) в высоконагруженной AI-системе?
3. Что такое Training-Serving Skew в контексте EDA и как его избежать?



Ключевые тезисы занятия

1. EDA позволяет продюсерам и консьюмерам масштабироваться независимо, сглаживая пиковые нагрузки через буфер брокера.
2. Выбирайте Fat Events (передача состояния), чтобы снизить нагрузку на БД и повысить автономность сервисов.
3. Запускайте переобучение по триггерам вместо cron



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

